IT5008: Tutorial 2 — Relational Schema & Integrity Constraints

Pratik Karmakar

School of Computing,
National University of Singapore

AY26/27 S1

The Intern's Schema (Book Exchange, Recap)

Same scenario as Tutorial 1, but this time we work against the **alternative schema** the intern wrote — flat tables, no ER diagram. The tables relevant to today's discussion:

- **student**(name, email, year, faculty, department, graduate)
- **book**(title, format, pages, language, author, publisher, pubdate, isbn10 *(unique)*, isbn13)
- **copy**(owner, book, copy, available) — `book` references `book(isbn13)`
- **loan**(borrower, owner, book, copy, borrowed, returned) — references `copy`

We'll assume Q1–Q2 (DDL/DML, done before class) are already sorted, and pick up at **Q3–Q4**, the questions marked for in-class discussion.

What Does DEFERRABLE Mean?

Only UNIQUE, PRIMARY KEY and FOREIGN KEY constraints can carry one of three qualifiers:

- NOT DEFERRABLE — the **default**. Always checked *immediately*, i.e. right after the statement that could violate it.
- DEFERRABLE INITIALLY IMMEDIATE — deferrable, but checked immediately unless a transaction says otherwise.
- DEFERRABLE INITIALLY DEFERRED — checked only at COMMIT / END TRANSACTION, unless overridden.

"Immediate" vs. "deferred" is about *when* the check runs: after every INSERT/UPDATE/DELETE, or once at the end of the whole transaction.

- CHECK constraints **cannot** be deferred in current PostgreSQL — only unique/PK/FK can.
- NUNStASchema.sql marks some FKs DEFERRABLE on purpose — that's what today's example exploits.

A Motivating Example

First, insert a copy of the new book, owned by Tikki:

```
1 INSERT INTO copy VALUES (
2   'tikki@gmail.com', '978-0321197849', 1, 'TRUE'
3 );
```

Now copy has a row whose book column is only valid *because* that ISBN-13 exists in book. What happens if we then try to delete *both* the book and its copy, in the same transaction?

```
1 BEGIN TRANSACTION;
2   SET CONSTRAINTS ALL <<IMMEDIATE|DEFERRED>>;
3   DELETE FROM book WHERE ISBN13 = '978-0321197849';
4   DELETE FROM copy WHERE book = '978-0321197849';
5 END TRANSACTION;
```

Transaction 1 — ALL IMMEDIATE

```
1  BEGIN TRANSACTION;
2    SET CONSTRAINTS ALL IMMEDIATE;
3    DELETE FROM book WHERE ISBN13 = '978-0321197849';   -- (*)
4    DELETE FROM copy WHERE book = '978-0321197849';
5  END TRANSACTION;
```

- Line (*) is checked *right away*: it would orphan the copy row we just inserted ⇒ the FK from copy to book is violated **immediately**.
- The statement fails and the transaction aborts — the second DELETE never even runs.
- ALL IMMEDIATE is the default anyway, so dropping that line changes nothing: it fails the same way.
- If a transaction is left hanging after an error, remember to issue END TRANSACTION (or ROLLBACK) yourself.

Transaction 2 — ALL DEFERRED

```
1  BEGIN TRANSACTION;
2    SET CONSTRAINTS ALL DEFERRED;
3    DELETE FROM book WHERE ISBN13 = '978-0321197849';   -- (*)
4    DELETE FROM copy WHERE book = '978-0321197849';     -- (**)
5  END TRANSACTION;
```

- After (*): the FK check is *postponed*. The database is momentarily **inconsistent** — a copy row pointing at a book that no longer exists.
- Run the two SELECTs from the tutorial sheet between (*) and (**) and you'd see exactly that: book empty, copy still there.
- After (**): the dangling copy row is gone too.
- At END TRANSACTION, the FK is finally checked — and by now it holds, so the transaction **commits**.

Why Bother Deferring?

- Some consistent *end states* are only reachable by passing through an inconsistent *intermediate* state — deferring lets the database tolerate that, as long as everything is fixed up by `COMMIT`.
- Checking eagerly (the default) is safer for accidents, but it forces you to order statements very carefully, or use extra tricks (temporary NULLs, deleting children before parents, . . .).
- Deferring only postpones *when* a constraint is checked — it never disables the constraint.
- Useful transaction control: `BEGIN`, `COMMIT`, `SAVEPOINT s`, `ROLLBACK TO s`.

`copy.available`: Data We Don't Need to Store

A copy is unavailable *exactly when* it's on loan and not yet returned — that's derivable from `loan`, so storing it again in copy is redundant (and can drift out of sync).

```
1 SELECT owner, book, copy, returned
2 FROM loan
3 WHERE returned ISNULL;          -- currently unavailable copies
```

Once we're convinced, drop the column:

```
1 ALTER TABLE copy
2 DROP COLUMN available;
```

Getting available Back — as a View

If client code still expects an `available` column, reconstruct it with a view instead of storing it:

```
1  CREATE OR REPLACE VIEW copy_view (owner, book, copy, available)
       AS (
2    SELECT DISTINCT c.owner, c.book, c.copy,
3      CASE WHEN EXISTS (
4        SELECT * FROM loan l
5        WHERE l.owner = c.owner AND l.book = c.book
6          AND l.copy = c.copy AND l.returned ISNULL
7      ) THEN 'FALSE' ELSE 'TRUE' END
8    FROM copy c
9  );
```

`SELECT * FROM copy_view;` now reads just like the old table did.

Is copy_view Updatable?

```
1 UPDATE copy_view
2 SET owner = 'tikki@google.com'
3 WHERE owner = 'tikki@gmail.com';
```

- In principle, every column *except* the derived `available` should be updatable — `owner`/`book`/`copy` map straight back to the underlying table.
- In practice, PostgreSQL won't do this automatically for a view built on `DISTINCT` + a correlated `EXISTS` subquery: it isn't "simple" enough to be auto-updatable.
- Fix: program the behaviour explicitly with an `INSTEAD OF` trigger, or an unconditional `ON UPDATE DO INSTEAD` rule — triggers are a second-half-of-semester topic.
- Drop the view once it's no longer needed: `DROP VIEW copy_view;`

`student(department, faculty)`: One Fact, Stored Twice

`department` **determines** `faculty`: every row with the same `department` must carry the same `faculty`. This is a **functional dependency** (department $\rightarrow$ faculty), formalized properly later in the course.

- Storing `faculty` on every `student` row repeats the same (department, faculty) pair once per student.
- That's redundant, and it opens the door to inconsistency: nothing stops two students in `'CS'` from being given *different* faculties by mistake.
- Fix: store the (department, faculty) mapping **once**, in its own table; `student` keeps only `department` and looks the faculty up via a foreign key.

The Refactor in SQL

```sql
CREATE TABLE department (
  department VARCHAR(32) PRIMARY KEY,
  faculty    VARCHAR(62) NOT NULL
);

INSERT INTO department
  SELECT DISTINCT department, faculty
  FROM student;

ALTER TABLE student
DROP COLUMN faculty;

ALTER TABLE student
ADD FOREIGN KEY (department) REFERENCES department(department);
```

Order matters: populate `department` from the existing data *before* dropping `faculty` from `student`, or the mapping is lost.

Questions?
Drop a mail at: pratik.karmakar@u.nus.edu